

Answers for part 2

1. What is the difference between a class and an object?
 - A class is the framework of an object, a class may define its methods or values stored inside the object. The object is then made, using the class, into an instance and stored inside memory.
2. What is inheritance? What is the Python syntax for inheritance?
 - Inheritance is something we use when we have subclasses or use an class hierarchy. When we want to inherit something from another class, lets say an method, we may define our own behaviour, or call `super.funcname()` to use the original from our mother class
3. What is the difference between a has-a and an is-a relationship?
 - Has-a is what we call composition and is-a is what we call inheritance. A good example of this is a car HAS an engine and a dog IS an animal. The main difference here is that when your object is-a relationship with another class, then all of the attributes the other class has gets passed over, allowing you to add more attributes in your own class. Meanwhile an has-a relationship contains another class inside its own class. This also allows for you to add your own attributes or even more classes, I just wanted to illustrate that both methods do the same thing, only the logic of when you apply it is different. Thus has-a and is-a relationship.
4. What is encapsulation? How is encapsulation handled in Python?
 - This is something we use when we want to bundle data or methods together, and make access restricted. The way we achieve this is by using naming to indicate what attributes should be accessible. For example `self.name` is accessible but `self._age` is protected. We can even go an step further and do `self.__age` whereby the double underscore makes it private and changes its name internally to `_classname__age`. By doing this we can use getters and setters to manipulate or get data from our class, safely.
5. What is polymorphism?
 - Its an technique used when multiple objects have the same function, for example `update()`. We can group all of them together and call `update()`, but they all operate with their own behaviour. That way, we can call `update` on an group inside a game loop, with only one line of code. And still update each object with its own special update protocol.